

CAHIER DES CHARGES

Outil d'audit de repositories & monitoring des KPIs développeur

Version 1.0 • 2026

1. Contexte

Dans le cadre du développement et du maintien de projets logiciels, les équipes techniques produisent un volume croissant de code réparti sur de nombreux repositories. Sans outillage adapté, il est difficile pour les responsables et tech leads de vérifier que les bonnes pratiques sont respectées : qualité du code, couverture de tests, sécurité des dépendances, lisibilité des commits, etc.

L'entreprise souhaite se doter d'un outil centralisé permettant d'auditer automatiquement ses repositories et d'afficher des indicateurs clés (KPIs) sous forme de tableaux de bord, afin de piloter la qualité logicielle et d'identifier les axes d'amélioration.

2. Besoin / Problématique

Actuellement, les données qualité sont dispersées entre plusieurs outils (GitHub/GitLab, SonarQube, Dependabot, etc.) sans vue consolidée. Les responsables n'ont pas de tableau de bord unique pour évaluer les pratiques de développement.

Problématique : Comment mettre en place un outil d'audit automatisé permettant de récupérer les données des repositories, d'analyser les KPIs de qualité et d'afficher les résultats sous forme de tableaux de bord afin de vérifier que les développeurs respectent les bonnes pratiques ?

3. Objectifs

Le projet a pour objectif de mettre en place un outil d'audit et de monitoring permettant :

- de connecter et récupérer automatiquement les données des repositories (GitHub, GitLab, Bitbucket, etc.) ;
- d'analyser les métriques de qualité et de bonnes pratiques ;
- d'afficher les KPIs sous forme de tableaux de bord clairs et filtrables ;
- de comparer les indicateurs entre projets, équipes ou périodes ;
- d'alerter en cas de non-respect des seuils définis ;
- de générer des rapports d'audit exportables ;
- de centraliser le suivi de la qualité logicielle en un point unique.

4. Utilisateurs

L'outil sera utilisé par différents profils au sein de l'organisation :

Utilisateur	Besoins principaux
Chef de projet / DSI	Piloter les audits, suivre les KPIs globaux, exporter les rapports
Développeur	Consulter ses propres indicateurs, comprendre les points d'amélioration
Tech Lead / Architecte	Analyser les standards de qualité par équipe ou projet
Responsable Qualité	Définir les seuils acceptables et alertes sur les KPIs
Administrateur	Gérer les accès, configurer les sources de repositories, paramétrer l'outil

5. Fonctionnalités

5.1 Connexion aux repositories

Le système devra permettre :

- de se connecter aux plateformes Git (GitHub, GitLab, Bitbucket) via API ou OAuth ;
- de sélectionner les organisations, groupes ou repositories à auditer ;
- de programmer des synchronisations automatiques (ex. : quotidienne, hebdomadaire) ;
- de gérer plusieurs comptes ou organisations simultanément.

5.2 Collecte et analyse des données

Le système devra être capable d'extraire et analyser les données suivantes :

- historique des commits (messages, fréquence, auteurs) ;
- branches et politique de gestion des branches ;
- pull requests / merge requests (nombre, durée, taux d'acceptation) ;
- couverture de tests (si intégration CI disponible) ;
- qualité du code (duplication, complexité, respect des linters) ;
- vulnérabilités et état des dépendances ;
- présence et qualité de la documentation (README, CHANGELOG, docs) ;
- issues ouvertes et leur ancienneté.

5.3 Tableau de bord et monitoring des KPIs

Le système devra offrir une interface de monitoring présentant :

- un tableau de bord global par organisation ou équipe ;
- une vue détaillée par repository ;
- des graphiques d'évolution dans le temps ;
- des filtres par période, équipe, repository ou développeur ;
- un système de scoring ou de notation par projet.

5.4 KPIs analysés

Les indicateurs suivants devront être mesurés et affichés :

KPI	Description	Seuil cible
Couverture de tests	Taux de code couvert par des tests unitaires / intégration	> 80 %
Taux de code dupliqué	Pourcentage de code dupliqué détecté (DRY)	< 5 %
Complexité cyclomatique	Mesure de la complexité logique des fonctions	< 10 par fonction
Nombre de vulnérabilités	Faillles de sécurité détectées dans le code ou les dépendances	0 critique, 0 haute
Nombre de issues / bugs ouverts	Tickets ouverts non résolus dans le repository	Tendance décroissante

KPI	Description	Seuil cible
Ratio PR / Merge	Proportion de pull requests acceptées vs rejetées	> 70 % acceptées
Fréquence de commit	Activité moyenne de contribution sur la période	Cohérent selon l'équipe
Qualité des messages de commit	Respect de conventions (Conventional Commits, etc.)	100 % conformes
Présence de documentation	Existence d'un README, d'un changelog, de docs techniques	Obligatoire
Dépendances obsolètes	Librairies non mises à jour ou dépréciées	0 critique

5.5 Alertes et seuils

Le système devra permettre :

- de définir des seuils d'alerte personnalisables par KPI ;
- d'envoyer des notifications (email, webhook, Slack) en cas de dépassement ;
- de signaler visuellement les repositories hors normes dans le tableau de bord.

5.6 Rapports d'audit

Le système devra permettre :

- de générer des rapports d'audit par repository ou par équipe ;
- d'exporter les rapports aux formats PDF et CSV ;
- de planifier l'envoi automatique de rapports périodiques.

5.7 Gestion des accès

Le système devra permettre :

- de gérer les comptes utilisateurs avec authentification sécurisée ;
- de définir des rôles et droits d'accès (admin, responsable, développeur) ;
- de restreindre la visibilité des données selon le périmètre de chaque utilisateur.

6. Contraintes

6.1 Techniques

- Le système devra être accessible via un navigateur web (interface responsive).
- L'architecture devra supporter plusieurs sources de repositories simultanément.
- Les appels API aux plateformes Git devront respecter les limites de rate-limiting.
- Le stockage des données collectées devra être géré par une base de données dédiée.
- Le système devra être déployable sur un environnement cloud ou on-premise.

6.2 Sécurité

- Les tokens d'accès aux APIs Git devront être stockés de manière sécurisée (chiffrés).
- L'accès à l'outil devra être protégé par une authentification (SSO ou login local).
- Les données collectées ne devront pas être accessibles sans droits appropriés.
- Les échanges devront être chiffrés (HTTPS).

6.3 Performance

- Les audits devront pouvoir être lancés en tâche de fond sans bloquer l'interface.
- L'affichage des tableaux de bord devra être fluide même avec de nombreux repositories.
- Le système devra gérer la montée en charge (ajout de nouveaux repositories et utilisateurs).

6.4 Documentation

L'ensemble du système devra être documenté. La documentation devra inclure :

- l'architecture technique du système ;
- le guide d'installation et de déploiement ;
- le guide d'utilisation pour chaque profil utilisateur ;
- les procédures de configuration des connexions aux repositories ;
- la description de chaque KPI et de son mode de calcul.

6.5 Évolutivité

- L'architecture devra permettre l'ajout de nouveaux KPIs sans refonte majeure.
- Le système devra pouvoir intégrer de nouvelles sources (Jira, SonarQube, CI/CD, etc.).
- La solution devra évoluer avec la croissance du nombre de repositories et d'utilisateurs.

6.6 Choix techniques

Aucune technologie particulière n'est imposée. Les choix devront être cohérents avec :

- les besoins fonctionnels décrits ci-dessus ;
- la facilité de maintenance et de déploiement ;
- la sécurité et la performance ;
- l'évolutivité à long terme.

Les choix techniques devront être justifiés dans le dossier de conception.